

YouTube Data Engineering Project

In-Depth Technical Documentation & Pipeline Walkthrough

1. Project Goal & Data Architecture

The objective was to create a scalable data lake on AWS following the **Medallion Architecture** to process YouTube trending data. The pipeline handles daily regional CSV statistics and JSON category metadata.

Architecture Layers: *Landing Area (Raw) → Cleansed/Enriched (Parquet) → Analytics/Reporting (Gold).*

2. Phase I: Ingestion & Infrastructure

AWS Account & IAM Setup

- Configured a Free Tier AWS account.
- Created an IAM user with programmatic access to enable local CLI interaction.
- Installed and configured AWS CLI with access keys.

Data Landing Zone (S3)

Created the raw bucket `yt-de-project-raw-dev`. Regional CSV files were uploaded using **Hive-style partitioning** to enable efficient partition pruning in Glue/Athena.

```
# Example Partitioning CLI Command
aws s3 cp CAvideos.csv s3://yt-de-project-raw-dev/youtube/raw_statistics/region=ca/
aws s3 cp . s3://yt-de-project-raw-dev/youtube/raw_statistics_reference_data/ --recursive --exclude
"*" --include "*.json"
```

3. Phase II: Semi-Structured Data ETL (Lambda)

The JSON Challenge

The raw JSON files were nested within an `items` array, making them unreadable for standard Athena SQL queries (causing `HIVE_CURSOR_ERROR`).

Serverless Transformation Logic

A Python Lambda function was deployed using the `awswrangler` and `Pandas` libraries.

- Flattening:** Used `pd.json_normalize()` to extract the nested schema.
- Schema Enforcement:** Explicitly cast the `id` field to `bigint` to ensure join compatibility.

- **Automation:** Configured an S3 trigger (`ObjectCreated`) for the specific JSON prefix and suffix.

```
# Core Transformation Snippet
df_raw = wr.s3.read_json(f's3://{bucket}/{key}')
df_flattened = pd.json_normalize(df_raw['items'])
wr.s3.to_parquet(df=df_flattened, path=s3_cleansed_path, dataset=True, database=db, table=table,
mode='append')
```

4. Phase III: Bulk Data ETL (Glue PySpark)

Distributed Processing for CSVs

Used **AWS Glue ETL** to handle the regional trending statistics. This allowed for scalable processing of larger datasets compared to Lambda.

Optimization	Implementation Details
Predicate Pushdown	Filtered specific regions (ca, gb, us) at the S3 source level to reduce scan costs.
Schema Evolution	Used <code>ApplyMapping</code> to cast views, likes, and comment counts to <code>bigint</code> .
Small File Problem	Used <code>.coalesce(1)</code> to group data into optimal file sizes for the Parquet sink.

5. Phase IV: Analytics & Join Layer

The Gold Layer

A final Glue Job joined the Cleansed Statistics and Category Reference data on `category_id` and `id`. The resulting dataset was stored in the `yt-de-project-analytics-dev` bucket and registered in `yt-de-project-analytics-db` for SQL consumption.

Validation Query (Athena)

```
SELECT a.title, a.category_id, b.snippet_title
FROM "yt-de-project-raw-db"."raw_statistics" a
INNER JOIN "yt-de-project-cleansed-db"."cleaned_statistics_reference_data" b
ON a.category_id = b.id;
```

Result: Successfully built a 100% serverless, automated data pipeline capable of transforming raw data into high-performance analytical assets.